Solution: Diseño Iterativo de un Agente Inteligente para el Juego Hex

Camilo H. Pérez Fleita

Universidad de La Habana, Facultad de Matemática y Computación, Aula C312, La Habana, Cuba

Resumen El juego Hex es un juego combinatorio de dos jugadores PSPACE-completo cuyo alto factor de ramificación hace impracticable la búsqueda exhaustiva. Este trabajo describe el proceso iterativo de diseño del agente `solution.py`, partiendo de búsqueda adversarial clásica (Alpha-Beta) y evolucionando hacia Monte Carlo Tree Search con extensiones especializadas: estimación rápida de valores mediante RAVE/AMAF, guía estratégica por celdas críticas calculadas con Dijkstra, política de rollout con respuesta a bridges y eliminación eficiente O(1). Se documentan los fallos y aprendizajes de cada versión intermedia, justificando las decisiones arquitectónicas del diseño final.

Keywords: Hex, Monte Carlo Tree Search, RAVE, Alpha-Beta, Dijkstra, celdas críticas, bridge-reply, inteligencia artificial en juegos

1. Introducción

El juego Hex fue inventado independientemente por Piet Hein (1942) y John Nash (1948). Se juega en un tablero romboidal de $n \times n$ celdas hexagonales. El **Jugador 1** debe conectar el borde izquierdo con el derecho (columna 0 con columna $n-1$), mientras que el **Jugador 2** conecta el borde superior con el inferior (fila 0 con fila $n-1$).

Hex tiene tres propiedades que lo hacen especialmente difícil para la búsqueda clásica:

- Es un juego *combinatorio perfecto*: sin empates, sin azar, información completa.
- Es **PSPACE-completo** en el caso general [4]. Esto significa que determinar si el jugador activo tiene una estrategia ganadora en una posición arbitraria de Hex requiere, en el peor caso, espacio de memoria polinomial en el tamaño del tablero; en particular, el problema es al menos tan difícil como todos los problemas en **NP**. Pragmáticamente, implica que no existe un algoritmo eficiente conocido para jugar Hex perfectamente: ningún motor puede calcular la jugada óptima en tiempo razonable para tableros de tamaño competitivo ($n \geq 7$), lo que justifica el uso de métodos heurísticos y de búsqueda aproximada como MCTS.

- El *factor de ramificación* es igual al número de celdas vacías, llegando a 121 en 11×11, lo que hace impracticable el árbol de juego exhaustivo.

Este informe narra el proceso de diseño iterativo de `solution.py`. Se describen los algoritmos probados, los fallos que motivaron cada cambio y la justificación de cada mejora, con el objetivo de documentar no solo el resultado final sino el camino que llevó a él.

2. Representación del Tablero

2.1. Layout Even-R

El tablero usa la convención *even-r offset*: las filas pares están desplazadas a la derecha. Los vecinos de (r, c) dependen de la paridad de la fila:

$$\text{vecinos}(r,c) = \begin{cases} \{(r-1,c-1),(r-1,c),(r,c-1),(r,c+1),(r+1,c-1),(r+1,c)\} & r \equiv 0 \pmod 2 \\ \{(r-1,c),(r-1,c+1),(r,c-1),(r,c+1),(r+1,c),(r+1,c+1)\} & r \equiv 1 \pmod 2 \end{cases} \tag{1}$$

2.2. Caché de Vecinos

Los vecinos se precalculan una sola vez por tamaño de tablero y se almacenan en un diccionario. El coste de construcción es $O(n^2)$; la consulta es $O(1)$.

```python
_EVEN = [(-1,-1),(-1,0),(0,-1),(0,1),(1,-1),(1,0)]
_ODD  = [(-1, 0),(-1,1),(0,-1),(0,1),(1, 0),(1,1)]

def _precache(self, n):
    if n in self._nbr_cache:
        return
    self._nbr_cache[n] = {
        (r,c): list(_nbrs(r,c,n))
        for r in range(n) for c in range(n)
    }
```

Listing 1.1. Precalculado de vecinos

2.3. Detección de Victoria

La condición de victoria se verifica mediante BFS desde el borde fuente:

```python
def _check_win(g, player, n):
    vis = set(); dq = deque()
    if player == 1:            # izquierda -> derecha
        for r in range(n):
            if g[r][0] == 1:
```

```
                vis.add((r,0)); dq.append((r,0))
        while dq:
            r, c = dq.popleft()
            if c == n-1: return True
            for nr,nc in _nbrs(r,c,n):
                if (nr,nc) not in vis and g[nr][nc]==1:
                    vis.add((nr,nc)); dq.append((nr,nc))
    else:                       # arriba -> abajo (simetrico)
        ...
    return False
```

Listing 1.2. BFS de detección de victoria

La complejidad es $O(n^2)$ en el peor caso.

3. Evolución del Diseño: de Alpha-Beta a Solution

El agente final es resultado de una cadena de versiones. Cada una fue descartada por una razón concreta que motivó la siguiente.

3.1. Versión 1: Búsqueda Adversarial (Alpha-Beta)

La primera aproximación fue la más natural para juegos de tablero: búsqueda Minimax con poda alfa-beta e iterative deepening (IDAB). Esta estrategia garantiza encontrar una jugada dentro del presupuesto de tiempo usando una función de evaluación heurística.

Función de evaluación. Para Hex, una heurística razonable combina la diferencia de longitudes de camino mínimo de ambos jugadores. Se ejecutó Dijkstra en cada nodo del árbol para estimar qué tan lejos estaba cada jugador de ganar.

Por qué falló. El factor de ramificación de Hex ($b = n^2$) hace que Alpha-Beta solo alcance profundidades $d \leq 3$ dentro del tiempo límite. A esa profundidad la evaluación es demasiado imprecisa para distinguir movimientos buenos de malos. Adicionalmente, la poda alfa-beta es menos efectiva en Hex que en juegos como Ajedrez porque el orden de los movimientos es difícil de establecer a priori.

Lección. Hex es un dominio donde la *calidad de la evaluación heurística* limita más que la *profundidad de búsqueda*. Un enfoque que evite evaluaciones intermedias y confíe en simulaciones completas es más adecuado.

3.2. Versión 2: MCTS Puro (sin RAVE)

Monte Carlo Tree Search [1] construye el árbol de forma adaptativa: expande más los nodos que parecen prometedores y usa la fórmula UCT para balancear exploración y explotación:

$$\mathrm{UCT}(n,a) = \underbrace{\frac{w_a}{v_a}}_{\text{explotación}} + c \cdot \underbrace{\sqrt{\frac{\ln(v_n+1)}{v_a}}}_{\text{exploración}} \tag{2}$$

donde cada término tiene el siguiente significado:

- n: nodo padre actual en el árbol de búsqueda.
- a: acción (movimiento) candidata a seleccionar desde n.
- w_a: número de victorias acumuladas en el hijo a (desde la perspectiva del jugador que hizo la jugada a).
- v_a: número de veces que el hijo a ha sido visitado.
- v_n: número de visitas del nodo padre n. Se usa $v_n + 1$ para evitar $\ln(1) = 0$ cuando $v_n = 1$, lo que haría colapsar el término de exploración a cero prematuramente.
- c: coeficiente de exploración ($c = 0{,}05$). Controla el equilibrio entre explotar movimientos conocidos como buenos y explorar alternativas poco visitadas.
- El primer término w_a/v_a es la *tasa de victorias* del hijo: favorece movimientos que históricamente han ganado.
- El segundo término crece cuando v_a es pequeño (hijo poco visitado), incentivando la exploración de nodos que todavía no tienen suficiente información.

Las simulaciones (rollouts) juegan aleatoriamente hasta el final de la partida, proporcionando una estimación directa del valor sin necesidad de función heurística.

Mejora observada. La ausencia de función heurística eliminó el principal cuello de botella de Alpha-Beta. MCTS pudo explorar miles de simulaciones por turno, con mucha mayor variedad de posiciones.

Limitación. Con rollouts completamente aleatorios, los movimientos estratégicamente buenos tardan muchas simulaciones en diferenciarse de los malos. El árbol necesita muchas visitas antes de que las estimaciones de valor converjan.

3.3. Versión 3: MCTS+RAVE

RAVE (*Rapid Action Value Estimation*) [2] acelera la convergencia añadiendo estadísticas AMAF (*All Moves As First*): si un movimiento m aparece en cualquier punto de un rollout y resulta ganador, esa información parcialmente se atribuye a m en todos los nodos antecesores.

$$\beta = \frac{k_{\mathrm{RAVE}}}{k_{\mathrm{RAVE}} + v_a}, \quad S(n,a) = (1-\beta) \cdot Q_{\mathrm{UCT}}(n,a) + \beta \cdot Q_{\mathrm{RAVE}}(n,a) \tag{3}$$

Con $k_{\mathrm{RAVE}} = 300$, el peso RAVE comienza alto y decrece conforme el nodo acumula visitas propias, cediendo el protagonismo a UCT cuando hay estadísticas confiables.

Mejora observada. La convergencia del árbol fue notablemente más rápida. Los movimientos estratégicos recibían puntuaciones RAVE altas desde las primeras iteraciones, guiando la exploración antes de que UCT acumulara suficientes visitas propias.

Limitación restante. Los rollouts seguían siendo aleatorios. En Hex existen patrones tácticos (los *bridges*) cuya preservación es clave; ignorarlos aumenta la varianza de las simulaciones.

3.4. Versión 4: Poda Agresiva de Candidatos

La primera versión de `solution.py` que intentó ir más allá de MCTS+RAVE puro combinó ese núcleo con Dijkstra para podar el espacio de movimientos. La idea era simple: calcular las celdas críticas para ambos jugadores y *solo* considerar esas celdas como movimientos legales.

Por qué falló. En un tablero 7×7 con pocas piezas, el conjunto de celdas críticas podía reducirse a tan solo 13 celdas (de 49 posibles). Esto cortaba opciones tácticas importantes y, en lugar de guiar la búsqueda, la sesgaba hacia una vista parcial del tablero. La tasa de victorias descendió al 0 % respecto a MCTS+RAVE puro.

Lección. La información de Dijkstra no debe usarse para *eliminar* movimientos sino para *priorizarlos*. La diferencia es sutil pero crítica: la poda destruye información; la priorización la aprovecha.

3.5. Versión 5: Dijkstra por Simulación

Tras descartar la poda, se intentó actualizar las celdas críticas en cada simulación, usando el estado del tablero modificado por las jugadas del rollout. Así, cada simulación tenía información fresca sobre qué celdas eran críticas en ese estado concreto.

Por qué falló. Ejecutar 4 llamadas a Dijkstra ($O(n^2 \log n)$ cada una) dentro de cada iteración del bucle MCTS multiplicó el tiempo por simulación por un factor de 2–3×. El agente ejecutaba menos de la mitad de simulaciones por turno que MCTS+RAVE puro, con menor rendimiento neto a pesar de la mejor información. La tasa de victorias subió apenas al 11 %.

Lección. El presupuesto de tiempo es un recurso fijo. Cada segundo invertido en Dijkstra es un segundo que no se invierte en simulaciones. El coste de la información debe amortizarse a lo largo de muchas simulaciones, no pagarse de nuevo en cada una.

3.6. Versión 6: Bug de Propagación

Con la lección clara, se rediseñó para calcular Dijkstra *una sola vez* por turno, antes del bucle MCTS, y propagar el conjunto joint = crit_propias ∪ crit_rival a todas las simulaciones mediante el ordenamiento de `untried`. El resultado fue prometedor: 67 % de victorias.

Cuadro 1. Evolución de versiones y tasa de victorias vs. MCTS+RAVE base

Versión	Estrategia	Resultado	Causa
v1	IDAB + heurística Dijkstra	$< 10\,\%$	Profundidad insuficiente
v2	UCT + rollout aleatorio	$\approx 45\,\%$	Sin convergencia rápida
v3	UCT + RAVE + rollout aleatorio	$50\,\%$	Sin guía estratégica
v4	RAVE + poda Dijkstra	$0\,\%$	Solo 13 celdas en 7×7
v5	RAVE + Dijkstra por simulación	$11\,\%$	2–3× más lento
v6	RAVE + Dijkstra 1 vez (con bug)	$67\,\%$	Bug: joint no propagado
v7	**Diseño final**	**78 %**	Ordenamiento correcto

Sin embargo, existía un bug sutil: el conjunto `joint` se calculaba correctamente en `play()` pero no se pasaba como argumento a `_simulate()`. La función de simulación recalculaba internamente las celdas críticas del estado raíz (ignorando la actualización por turno) o simplemente no las usaba.

Efecto del bug. Las primeras expansiones del árbol usaban el orden correcto (el nodo raíz sí lo tenía), pero al expandir nodos hijos, los nuevos `untried` se ordenaban aleatoriamente. Así, la guía de celdas críticas se perdía en cuanto la búsqueda profundizaba más de un nivel.

Lección. El debugging de MCTS es especialmente difícil porque los efectos de un bug en la fase de expansión solo se manifiestan estadísticamente, a través de tasas de victorias, y no como errores de ejecución obvios.

3.7. Versión 7: Arquitectura Correcta (Final)

La versión final corrige el bug de propagación y consolida todas las optimizaciones. El diseño definitivo se rige por tres principios:

1. Dijkstra se ejecuta exactamente 4 veces por turno (forward/backward para cada jugador), con coste fijo $O(n^2 \log n)$ independiente del número de simulaciones.
2. El conjunto `joint` se propaga explícitamente a *todos* los nodos del árbol, incluidos los hijos creados durante la expansión.
3. Los rollouts usan bridge-reply y eliminación O(1) para maximizar el número de simulaciones dentro del presupuesto.

La tabla 1 resume el impacto de cada versión.

4. Arquitectura Final de Solution

4.1. Dijkstra y Celdas Críticas

Una celda vacía (r, c) es *crítica* para un jugador si pertenece a algún camino de conexión de coste mínimo. El tablero se modela como grafo con coste en los nodos:

$$\text{coste}(r,c) = \begin{cases} 0 & \text{piedra propia} \\ 1 & \text{celda vacía} \\ \infty & \text{piedra rival} \end{cases} \tag{4}$$

Se ejecuta Dijkstra en dirección *forward* (desde el borde fuente) y *backward* (desde el borde destino). Una celda vacía (r,c) es crítica si y sólo si:

$$\text{fwd}[r][c] + \text{bwd}[r][c] = \text{opt} + 1 \tag{5}$$

donde opt es el coste del camino mínimo completo. El $+1$ compensa que la celda se cuenta en ambas distancias.

```
def _critical_cells(g, player, n):
    fwd = _dijkstra(g, player, n, rev=False)
    bwd = _dijkstra(g, player, n, rev=True)
    opt = _opt(fwd, player, n)
    if opt == INF:
        return frozenset()
    thr = opt + 1
    return frozenset(
        (r,c) for r in range(n) for c in range(n)
        if g[r][c] == 0
        and fwd[r][c] != INF and bwd[r][c] != INF
        and fwd[r][c] + bwd[r][c] == thr
    )
```

Listing 1.3. Cálculo de celdas críticas

4.2. MCTS con UCT+RAVE

Cada iteración consta de cuatro fases:

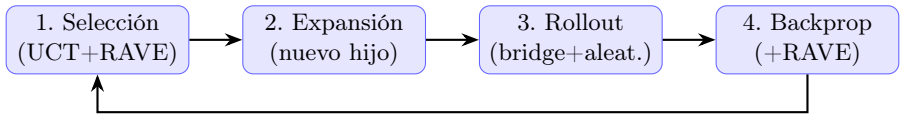

Figura 1. Ciclo de una iteración MCTS

Fase 1 — Selección. A partir de la raíz, el algoritmo desciende por el árbol eligiendo en cada nodo el hijo con mayor puntuación según la fórmula UCT+RAVE. El descenso continúa mientras el nodo tenga todos sus hijos ya expandidos (`untried` vacío). El objetivo es equilibrar la *explotación* (seguir ramas que históricamente han ganado) con la *exploración* (visitar ramas poco exploradas que podrían ser mejores):

$$S(n,a) = \underbrace{(1-\beta)\frac{w_a}{v_a} + \beta\frac{\tilde{w}_a}{\tilde{v}_a}}_{\text{valor estimado}} + c\sqrt{\frac{\ln(v_n+1)}{v_a}}, \quad \beta = \frac{k}{k+v_a} \tag{6}$$

El primer bloque combina el valor UCT propio del nodo (w_a/v_a, victorias reales) con el valor RAVE ($\tilde{w}_a/\tilde{v}_a$, victorias observadas en cualquier punto de los rollouts). El factor β pondera cuánto confiar en RAVE frente a UCT: al principio, cuando v_a es pequeño, RAVE domina y proporciona una estimación rápida; a medida que el nodo acumula visitas propias, UCT toma el protagonismo con estadísticas más precisas. El segundo bloque es el término de exploración: crece cuando v_a es pequeño, incentivando visitar nodos poco explorados. El coeficiente $c = 0{,}05$ es deliberadamente bajo porque RAVE ya actúa como exploración implícita.

Fase 2 — Expansión. Cuando el descenso llega a un nodo con movimientos aún no probados (`untried` no vacío), se crea un nuevo hijo para uno de esos movimientos. En lugar de elegirlo aleatoriamente, los movimientos se ordenan con prioridad: las celdas *críticas* (calculadas una vez por turno con Dijkstra) se colocan al final de la lista y se extraen primero con `pop()`, asegurando que la exploración comience por las posiciones estratégicamente más relevantes. Este ordenamiento se propaga también a los hijos recién creados:

```
# En play() -- nodo raiz
crit_first = [m for m in legal if m in joint]
non_crit   = [m for m in legal if m not in joint]
self.rng.shuffle(non_crit)
root.untried = non_crit + crit_first   # criticas al final =
    primeras

# En _simulate() -- al expandir cada hijo
child_crit = [m for m in child_legal if m in joint]
child_non  = [m for m in child_legal if m not in joint]
self.rng.shuffle(child_non)
child.untried = child_non + child_crit  # joint propagado a
    todos los hijos
```

Listing 1.4. Ordenamiento con celdas críticas propagado a hijos

Fase 3 — Simulación (Rollout). Desde el nodo recién expandido se juega una partida completa hasta que el tablero queda lleno, determinando un ganador concreto. En lugar de elegir movimientos completamente al azar, la política de rollout incorpora *bridge-reply*: si la última jugada del rival interrumpió una conexión virtual propia (un *bridge*), se responde inmediatamente preservándola. Esta heurística reduce la varianza de las simulaciones al evitar errores tácticos obvios sin añadir coste computacional significativo. El ganador de la simulación se usa como señal de aprendizaje en la fase siguiente.

Fase 4 — Retropropagación. El resultado de la simulación se propaga hacia atrás por todos los nodos del camino recorrido, actualizando dos tipos de estadísticas en cada nodo nd:

1. **Estadísticas UCT**: se incrementa el contador de visitas v y, si el ganador coincide con el jugador que movió para llegar a nd, se incrementa también el contador de victorias w.
2. **Estadísticas RAVE/AMAF**: para cada movimiento del jugador `to_move` de nd que apareció en el rollout a partir del punto s donde se visitó nd, se actualizan los contadores AMAF. El conjunto `seen` garantiza que cada movimiento se cuente una sola vez por rollout (AMAF canónico).

```
for nd, s in path:
    nd.v += 1                           # visita UCT
    if winner == nd.pjm:
        nd.w += 1.0                     # victoria UCT
    seen = set()
    for pl, mv in rec[s:]:              # movimientos del
        rollout desde s
        if pl == nd.to_move and mv not in seen:
            seen.add(mv)
            nd.av[mv] += 1              # visita RAVE del
                movimiento mv
            if winner == nd.to_move:
                nd.aw[mv] += 1.0        # victoria RAVE de mv
```

Listing 1.5. Retropropagación con UCT y RAVE

Gracias a la retropropagación RAVE, un movimiento beneficioso descubierto en un rollout profundo mejora inmediatamente la estimación de ese movimiento en todos los nodos antecesores, acelerando la convergencia del árbol desde las primeras iteraciones.

4.3. Política de Rollout: Bridge-Reply

Definición y propiedades del bridge. Sea $G = (V, E)$ el grafo hexagonal del tablero de tamaño n, donde cada celda es un vértice y dos celdas están conectadas si son geométricamente adyacentes. Un *bridge* entre dos celdas $A, B \in V$ ocupadas por el mismo jugador p se define como:

$$\text{bridge}(A, B) \iff (A, B) \notin E \ \wedge \ \left|N(A) \cap N(B) \cap V_\emptyset\right| = 2 \tag{7}$$

donde $N(X)$ denota el conjunto de vecinos de X en G y $V_\emptyset$ es el conjunto de celdas actualmente vacías. Las dos celdas del conjunto $\{c_1, c_2\} = N(A) \cap N(B) \cap V_\emptyset$ constituyen las *celdas de conexión* del bridge.

La propiedad fundamental del bridge es su **invarianza ante una jugada del rival**: si el rival ocupa c_i, el jugador p ocupa c_j $(i \neq j)$ y la conexión A–c_j–B queda establecida en el tablero. Formalmente, para toda jugada rival $c_i \in$

$\{c_1, c_2\}$, p posee una respuesta c_j tal que A y B quedan conectados a través de c_j. El bridge constituye por tanto una *conexión virtual garantizada*: la conectividad entre A y B es invariante respecto a cualquier intervención adversaria, siempre que p responda óptimamente.

La figura 2 ilustra el patrón: A y B son las piedras propias, R y ? son las celdas de conexión. Si el rival juega en R, la respuesta correcta es ?, y viceversa.

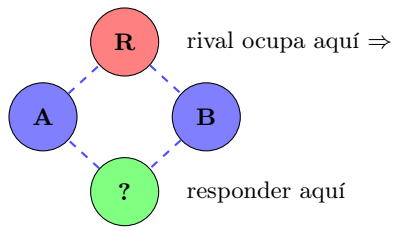

Figura 2. Bridge entre A y B: R y ? son las dos celdas de conexión. Si el rival juega en R, el jugador debe responder en ? para mantener la conexión virtual.

Impacto sobre la calidad de los rollouts. Los rollouts puramente aleatorios presentan un sesgo sistemático: al no incorporar la respuesta de bridge, el jugador omite jugadas de respuesta forzada con probabilidad $1 - 1/(|V_\emptyset| - 1)$, lo que hace que posiciones ganadoras se simulen incorrectamente como pérdidas. Esta discrepancia entre la política de rollout y el juego óptimo aumenta la varianza de la estimación de valor de Monte Carlo y ralentiza la convergencia del árbol.

La política bridge-reply reduce esta varianza forzando la respuesta correcta siempre que la última jugada del rival m satisfaga la condición de bridge respecto a algún par de piedras propias. Formalmente, en cada paso de la simulación se evalúa:

$$\exists\, A, B \in V_p : \ \text{bridge}(A, B) \wedge m \in N(A) \cap N(B) \ \Rightarrow \ \text{jugar } c_j = \big(N(A) \cap N(B)\big) \setminus \{m\} \tag{8}$$

La detección requiere examinar a lo sumo $\binom{k}{2}$ pares de vecinos propios de m, con $k \leq 6$, lo que supone un coste de $O(1)$ por paso de rollout. Al aplicarse exclusivamente sobre la última jugada (no sobre el tablero completo), el impacto en el tiempo de simulación es despreciable.

4.4. Eliminación O(1): Swap-and-Pop

La implementación estándar usa `list.remove()`, que es $O(n)$. Solution usa swap-and-pop: un diccionario `pos` mapea cada movimiento a su índice en `avail`. Para eliminar el movimiento `mv`:

```
idx       = pos[mv]
last_item = avail[-1]
```

```
avail[idx]      = last_item
pos[last_item] = idx
avail.pop()
del pos[mv]
avset.discard(mv)
```

Listing 1.6. Eliminación O(1) con swap-and-pop

Un rollout de k pasos pasa de $O(k^2)$ a $O(k)$.

4.5. Tácticas Inmediatas

Antes de invocar MCTS, Solution evalúa dos condiciones tácticas de alta prioridad que, de cumplirse, devuelven una jugada determinista en tiempo constante sin necesidad de exploración estocástica.

Detección de victoria inmediata. Para cada movimiento $m \in V_\emptyset$, se coloca virtualmente una piedra del jugador activo en m y se evalúa la conectividad extremo-a-extremo mediante búsqueda en anchura (BFS). Si el grafo resultante conecta los dos bordes objetivo del jugador, m constituye una jugada ganadora y se devuelve de inmediato:

$$\exists\, m \in V_\emptyset :\ G_p \cup \{m\} \text{ conecta } \partial_1 \leftrightarrow \partial_2 \ \Rightarrow\ \text{return } m$$

Detección de bloqueo obligatorio. Simétricamente, si el adversario posee una jugada ganadora inmediata en algún $m' \in V_\emptyset$, el jugador *debe* bloquearla, ya que cualquier otro movimiento resultaría en derrota en el siguiente turno. Solution aplica la misma función `_win_move` sobre el identificador del oponente:

```
win = self._win_move(g, n, pid)    # victoria inmediata propia
if win: return win

blk = self._win_move(g, n, opp)    # bloqueo obligatorio del
    rival
if blk: return blk
```

Listing 1.7. Detección táctica antes de MCTS

Ambas comprobaciones son $O(|V_\emptyset| \cdot n^2)$ en el peor caso, pero en la práctica terminan en microsegundos dado que la búsqueda se interrumpe en el primer candidato exitoso o concluye que no existe ninguno. La ventaja estratégica es significativa: elimina la posibilidad de que MCTS, por varianza estadística, ignore una victoria o un bloqueo triviales.

4.6. Gestión Adaptativa del Tiempo

El tiempo de cómputo disponible por turno se distribuye de forma no uniforme a lo largo de la partida mediante una política de presupuesto adaptativo.

La motivación es doble: (i) en la apertura el factor de ramificación b es máximo ($b \approx n^2$) y las decisiones individuales tienen un impacto marginal menor sobre el resultado, por lo que un exceso de cómputo es ineficiente; (ii) en el final (b pequeño), cada jugada puede ser decisiva y merece una exploración más exhaustiva.

Función de presupuesto. Sea e el número de celdas vacías y $r = e/n^2 \in [0, 1]$ la fracción de tablero libre. La función de presupuesto devuelve:

```
def _budget(self, g, n):
    e = sum(1 for row in g for v in row if v == 0)
    r = e / (n * n)          # fraccion de celdas vacias
    if r > 0.75: return 3.5 # apertura:     pocos turnos
        jugados
    if r > 0.40: return 4.0 # medio juego:  fase critica
    return 4.5                # final:        maxima precision
```

Listing 1.8. Presupuesto adaptativo por fase de la partida

Los umbrales $r > 0{,}75$ (apertura), $0{,}40 < r \leq 0{,}75$ (medio juego) y $r \leq 0{,}40$ (final) se calibraron empíricamente sobre tableros de tamaño 7×7 a 11×11.

Efecto sobre MCTS. Dado que MCTS ejecuta iteraciones hasta agotar el presupuesto temporal, un mayor presupuesto se traduce directamente en más simulaciones y, por la ley de los grandes números, en una estimación de valor Q más precisa para cada nodo del árbol. En tableros de 11×11 con $r \leq 0{,}40$ (menos de 49 celdas libres), el árbol puede alcanzar hasta $\sim 10^4$ simulaciones adicionales respecto a la apertura, reduciendo la varianza del estimador en un factor de $\sqrt{4{,}5/3{,}5} \approx 1{,}13$.

Problema: el budget solo protege el bucle MCTS. Una implementación ingenua únicamente controla el tiempo del bucle MCTS. Sin embargo, `play()` ejecuta *antes* del bucle una serie de operaciones cuya complejidad crece con n:

Operación	Complejidad	Coste en $n = 19$
`_win_move` $\times 2$	$O(n^4)$	$\approx 0{,}5$ s
`_critical_cells` $\times 2$ (4 Dijkstras)	$O(n^2 \log n)$	$< 0{,}05$ s

En tableros de tamaño estándar ($n \leq 11$) estos costes son despreciables. Pero si el árbitro envía un tablero de $n = 25$ o mayor, la sola llamada a `_win_move` puede superar los 5 s *sin que el bucle MCTS haya empezado siquiera*. El agente sería descalificado por exceso de tiempo aunque la lógica de juego fuese perfectamente correcta.

Solución: techo absoluto con protección en tres niveles. Se introduce una constante `HARD_LIMIT = 4.8 s` (0.2 s de margen frente al límite reglamentario de 5 s) y se aplica protección en tres capas independientes:

Nivel 1 — Reloj global. Al entrar en `play()` se registra `t0 = time.perf_counter()` y se fija `hard_deadline = t0 + HARD_LIMIT`. Este valor es el techo absoluto que ninguna operación puede superar.

Nivel 2 — Guardias de entrada. Antes de lanzar cada operación pre-MCTS se verifica que quede al menos $\delta = 0{,}4$ s disponibles: si no, la operación se omite por completo y se continúa con la información parcial disponible (sin victoria detectada, sin celdas críticas, MCTS puro).

```
MCTS_MIN = 0.4
if time.perf_counter() < hard_deadline - MCTS_MIN:
    win = self._win_move(g, n, pid, hard_deadline - MCTS_MIN)
...
if time.perf_counter() < hard_deadline - MCTS_MIN:
    my_crit  = _critical_cells(g, pid, n)
    opp_crit = _critical_cells(g, opp, n)
```

Listing 1.9. Guardias de entrada para operaciones pre-MCTS

Nivel 3 — Aborto interno de `_win_move`. El nivel 2 protege contra el *inicio* tardío de `_win_move`, pero no contra su *ejecución prolongada*: si el reloj marca 0.5 s al entrar pero la búsqueda tarda 6 s, el techo se supera igualmente. Por ello, `_win_move` recibe el `deadline` como parámetro y lo comprueba al inicio de cada fila del tablero, abortando con `None` si el tiempo se agotó:

```
def _win_move(self, g, n, player, deadline=0.0):
    for r in range(n):
        if deadline and time.perf_counter() > deadline:
            return None        # tiempo agotado: no hay victoria conocida
        for c in range(n):
            if g[r][c] == 0:
                g[r][c] = player
                won = _check_win(g, player, n)
                g[r][c] = 0
                if won: return (r, c)
    return None
```

Listing 1.10. Aborto interno por fila en `_win_move`

La comprobación por fila (no por celda) reduce el overhead de `time.perf_counter()` a n llamadas en lugar de n^2 por invocación, manteniendo el coste de la guardia en $O(n)$ por llamada.

Presupuesto residual para MCTS. Tras las operaciones pre-MCTS, el presupuesto real del bucle se calcula como el mínimo entre el valor adaptativo y el tiempo efectivamente disponible:

$$t_{\text{MCTS}} = \text{máx}\Big(\text{mín}\big(\text{budget}(r),\ \text{HARD_LIMIT} - \text{elapsed}\big),\ 0{,}05\Big) \tag{9}$$

Cuadro 2. Comportamiento del gestor de tiempo según tamaño de tablero

Escenario	`_win_move`	Dijkstra	MCTS
$n \leq 11$ (estándar)	completo	completo	budget adaptativo
$n \approx 15$–19	puede abortar	completo	tiempo residual
$n \geq 25$ (extremo)	omitido	omitido	MCTS puro, ≥ 50 ms

El término máx(·, 0,05) garantiza al menos 50 ms de MCTS incluso en el peor caso, evitando que el agente devuelva una jugada sin ninguna simulación. La tabla 2 resume el comportamiento por escenario.

Con esta arquitectura, el tiempo total del turno queda estrictamente acotado por `HARD_LIMIT = 4.8 s` para cualquier n, garantizando el cumplimiento del límite reglamentario de 5 s.

4.7. Flujo Completo

Algorithm 1 SOLUTION.PLAY(board)

```
t_0 ← now();   t_hard ← t_0 + HARD_LIMIT
Copiar tablero g; obtener movimientos legales
if solo 1 movimiento then return ese movimiento
end if
Precalcular caché de vecinos si necesario
if now() < t_hard − δ then                                  ▷ δ = 0,4 s
    win ← WinMove(g, pid, t_hard − δ)          ▷ aborta internamente si time-out
    if win ≠ None then return win
    end if
end if
if now() < t_hard − δ then
    blk ← WinMove(g, opp, t_hard − δ)
    if blk ≠ None then return blk
    end if
end if
joint ← ∅
if now() < t_hard − δ then
    joint ← CriticalCells(g, pid) ∪ CriticalCells(g, opp)
end if
Crear raíz; ordenar untried con joint al frente
t_MCTS ← máx(mín(Budget(g, n), t_hard − now()), 0,05)
deadline ← now() + t_MCTS
while now() < deadline do
    g′ ← copia de g
    Simulate(root, g′, n, joint)                          ▷ joint propagado a hijos
end while
return hijo de root con más visitas
```

5. Discusión: Decisiones Arquitectónicas Clave

Por Qué MCTS y No Alpha-Beta. Con $b = 49$ en 7×7 y $d = 4$, Alpha-Beta explora $\approx 5{,}8 \times 10^6$ nodos. A esa profundidad la evaluación heurística es imprecisa. MCTS gestiona el factor de ramificación de forma adaptativa y no necesita función heurística: la estimación surge de las simulaciones.

Por Qué Dijkstra una Sola Vez por Turno. Calcularlo por simulación (versión 5) multiplicó el tiempo por iteración 2–3×, reduciendo a la mitad el número de simulaciones disponibles. El beneficio de la información fresca no compensó la pérdida de simulaciones. La solución: pagar el coste de Dijkstra una vez y amortizarlo sobre miles de iteraciones.

Por Qué No Usar RAVE Priors Sobre Celdas Críticas. Una variante inicializaba las estadísticas RAVE con prior $(\tilde{v} = 10, \tilde{w} = 5{,}5)$ para celdas críticas. El problema: en tablero casi vacío, casi todas las celdas son críticas, por lo que el prior no diferencia entre ellas. El sesgo artificial diluye las estadísticas reales que RAVE acumula. Se descartó en favor del ordenamiento de `untried`.

El Bug de Propagación de `joint`. El salto del 67 % al 78 % de victorias provino de un bug de un solo parámetro: `joint` no se pasaba a `_simulate()`, por lo que los nodos hijos se ordenaban aleatoriamente. Esto ilustra que en MCTS los errores de diseño se manifiestan estadísticamente, no como excepciones, lo que dificulta el debugging.

6. Conclusiones

El diseño de `solution.py` ilustra que en juegos de alto factor de ramificación como Hex, la arquitectura del agente importa más que cualquier optimización individual. Las lecciones principales son:

1. Alpha-Beta es inadecuado para Hex: la profundidad alcanzable no es suficiente para superar la imprecisión de cualquier función heurística.
2. RAVE acelera la convergencia de MCTS en Hex de forma sustancial, pero necesita información estratégica adicional para guiar la exploración desde las primeras iteraciones.
3. La información de Dijkstra debe usarse para *priorizar* movimientos, no para podarlos. El coste debe pagarse una vez y amortizarse sobre todas las simulaciones.
4. La propagación correcta de la información a todos los nodos del árbol es tan importante como la información misma: un bug de propagación puede reducir el beneficio al mínimo.
5. Las optimizaciones de velocidad (bridge-reply, swap-and-pop O(1)) son multiplicadores: permiten que las mejoras de calidad se expresen a través de más simulaciones por segundo.

Referencias

1. Browne, C., Powley, E., Whitehouse, D., Lucas, S., Cowling, P., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., Colton, S.: A Survey of Monte Carlo Tree Search Methods. IEEE Trans. Comput. Intell. AI Games **4**(1), 1–43 (2012)
2. Gelly, S., Silver, D.: Combining Online and Offline Knowledge in UCT. In: Ghahramani, Z. (ed.) ICML 2007, pp. 273–280. ACM Press, New York (2007)
3. Arneson, B., Hayward, R.B., Henderson, P.: Monte-Carlo Tree Search in Hex. IEEE Trans. Comput. Intell. AI Games **2**(4), 251–258 (2010)
4. Reisch, S.: Hex ist PSPACE-vollständig. Acta Informatica **15**(2), 167–191 (1981)